DSP Playground

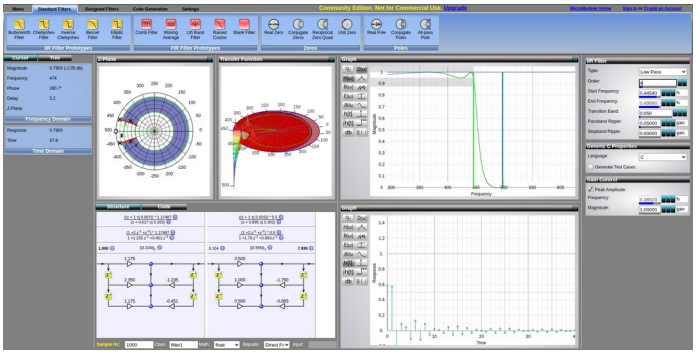
0.2.1 Release Notes

This version essentially troubleshoots the malfunctioning of the last version.

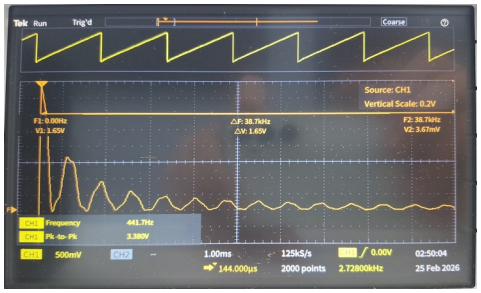
Problem description:
According to the code:

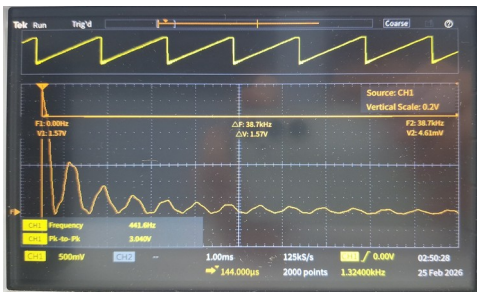
```
//calculate next value of the filter's output (y_n) according to the new measured input sample (x_n),
//the past values (filter.pState - stateVariables array) and the coefficients (filter.pCoeffs)
x_n = (float32_t)sawtooth_value;
arm_biquad_cascade_df1_f32(&filter, &x_n, &y_n, 1);
filteredWave_value = (uint8_t)y_n;
```

And the filter model:



The output should almost kill the whole spectrum after the fundamental frequency (440Hz). However, the result follows (sawtooth and filtered; PA4 and PA5, respectively):





Note that the spectrum barely changes.

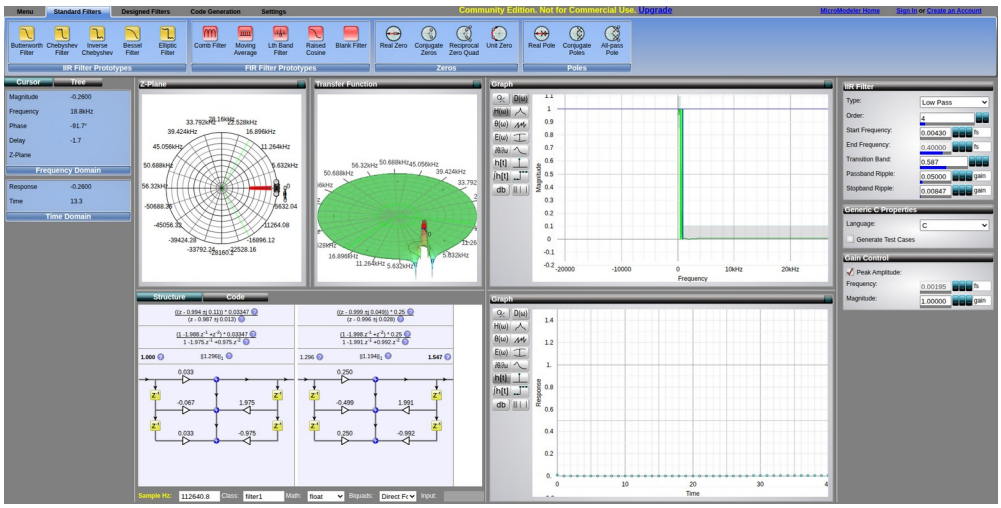
Troubleshooting:

First thing I wanted to make sure if the problem was not a bad implementation of the CMSIS DSP library. So I started inputting the ADC value to the filter instead of the DAC itself. The spectrum looked virtually the same.

After a thorougher analysis on the Micromodeler DSP software, I noticed that changing Sample Hz changed the cutoff frequency in the Graph window, but didn't change the coefficients, which is senseless. A different filter needs to have different coefficients. With the standard "1" value, the cutoff frequency couldn't be more than 500mHz.

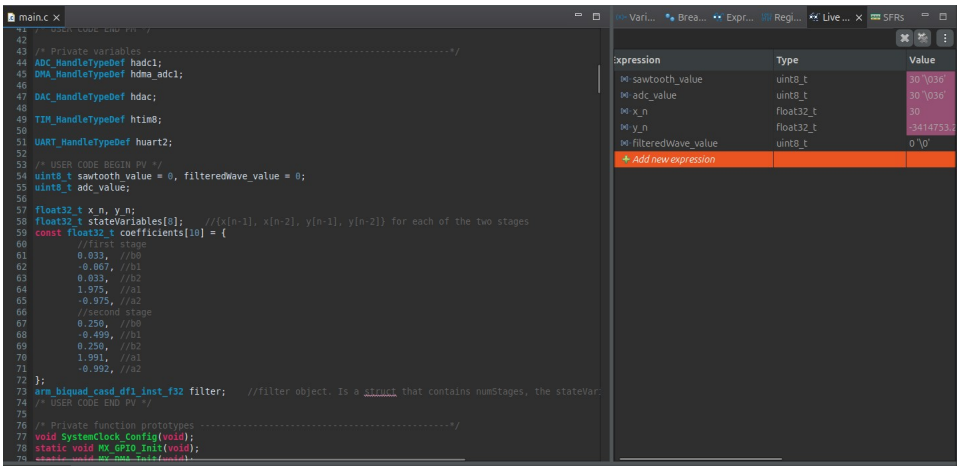
With a deeper research (eureka after reading SciPy documentation) I realized what I was doing wrong. In any IIR filter, one of the parameters is the sample rate with which the signal is acquired. That was the "Sample Hz" setting. It's not just a magnitude order settings as I had supposed. So it makes sense that changing Sample Hz changed the filter's properties, but not the coefficients.

Back to Micromodeler, I set the sample rate in use (filter not saved as a file because it required an account that I couldn't create due to an image loading failure in CAPTCHA):



Note that the transition window is at a much smaller frequency than the Nyquist frequency. This means the filter is bad designed because the sample rate is too high (~113kHz) compared to the cutoff frequency (~500Hz).

Even so, I decided to test the new coefficients. The speaker stood quiet. Using the debugger, I found out that *y_n* decreased indefinitely.



Before hunting any other bugs, the sample rate problem must be solved. This could just be a precision error, as the transition band is very narrow and at very low frequencies.